`RevealStarted` events are accessed and how the user is notified of these events. These two events do not have any parameters:

```
if(parseInt(result.receipt.status) == 1){
                if(result.logs.length > 0){
                  App.showNotification(result.logs[0].event);
                }
```

The next snippet is inside the `handleWinner()` function of app.js. It executes the `auctionEnd()` function, which in turn triggers the `AuctionEnded` event with two parameters: `winner` and `highestBid`. These parameters are logged as on-chain data and returned as results that are extracted by the following code. (The `winner` argument of `logs[0]` and `highestBid` argument of `logs[0]` are extracted for display in the UI.) Also note the `toNumber()` utility function that converts the `highestBid` argument (a 256-bit number) to a smaller 64-bit number:

```
return bidInstance.auctionEnd();
      }).then(function(res){
        var winner = res.logs[0].args.winner;
        var highestBid = res.logs[0].args.highestBid.toNumber();
```

These two snippets demonstrate how to access the event logs in transaction receipts and use them in your off-chain applications. This on-chain data is useful for real-time notifications, as illustrated in this demo, as well as for other purposes. The event logs are stored and indexed, and they can be extracted for offline data analytics based on topics and logs of immutable blockchain data.